

AI Master Project 6기

최종 발표

과제명 WOONG AI —Warehouse Ops Orchestration & Navigation
Governance AI
멘 티 정선웅, 11753
멘 토 이재영

발표 영상 10분 이내 | 시연 영상 별도 제출 5분 이내

프로젝트 개요

권장 2분 이내

1. 문제 정의 (Vision / Problem)

참고 운영 지식과 정책이 분산되어 있어,
질의·계산·정책 판단·승인까지 시간이 오래 걸리고
오류와 위험이 발생하는 문제를 해결하고자 했습니다.



비즈니스 임팩트

의사결정 지연, 재고 부족/과잉, 비효율적 자원 사용으로
운영 비용 증가 및 서비스 품질 저하가 발생합니다.

2. 핵심 기능 (Core Features)

자연어 질의를 이해해 필요한 도구와 정책을 연결하고,
계산·추천·시뮬레이션·승인까지 안전하게 수행합니다.



핵심 기술 스택

LangGraph, FastAPI, Vanilla JS SPA, FAISS, SQLite, SimPy(DES),
Azure OpenAI 호환 Gateway



P50과 P90이란? (쉽게 설명)

시뮬레이션을 여러 번 돌리면 결과가 조금씩 달라집니다.
P50은 '보통 이 정도예요', P90은 '상황이 안 좋으면 이 정도까지 갈 수 있어요'라는 뜻입니다.
예) 출고 지연이 P50 20분, P90 55분이면 → 절반은 20분 안팎, 90%는 55분 이내라는 뜻입니다.

핵심 구현 성과 (Key Implementation Results)



26개

업무 Intent 구현

참고 핵심 업무 26개
Intent를 분류·구현하여
정확한 Tool과 정책으로
연결합니다.



60/60

평가 Harness 통과

Agent·Tool·RAG·
Guardrail 평가 Harness
60개 항목을 100%
통과했습니다.



P50/P90

KPI 제공 (What-if 분석)

DES 반복 시뮬레이션 결과를
P50(보통)과 P90(나쁜 상황)
으로 제공하여 운영 변동성과
위험을 함께 비교할 수
있습니다.

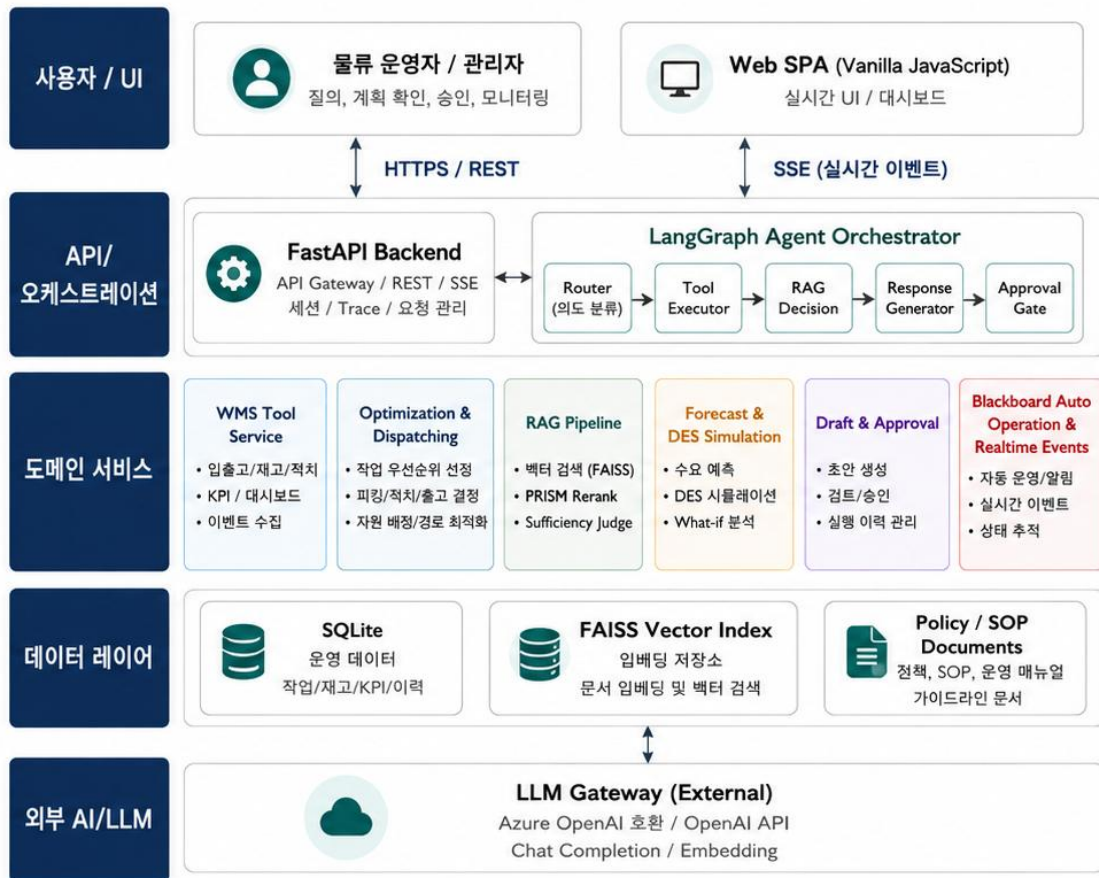
Key Message



LLM 자신이 계산하지 않고,
Tool·RAG·최적화·시뮬레이션·승인을 연결해
정확하고 안전한 운영 의사결정을 지원합니다.

기술 아키텍처

권장 4분 이내



기술 아키텍처 고려 사항

아키텍처 구성의 핵심 결정 사항과 주요 기술 선택 이유를 정리했습니다.

01 기술명: LangGraph + FastAPI + Azure OpenAI 호환 Gateway

선택 이유:

단순 Chain 방식으로는 라우팅, Tool 호출, 승인 흐름, 스트리밍 응답을 유연하게 제어하기 어려워 Agent Orchestration 구조를 선택했습니다.

핵심 활용:

Router → Tool Executor → RAG Decision → Response Generator → Approval Gate 흐름으로 구성하고, FastAPI 기반 REST/SSE로 UI와 연동했습니다.

02 기술명: RAG + FAISS + PRISM

선택 이유:

Fine-tuning보다 정책·SOP 문서를 빠르게 반영하고 근거 기반 답변을 제공하기 위해 RAG 방식을 채택했습니다.

핵심 활용:

FAISS 벡터 검색 후 PRISM Rerank와 Sufficiency Judge를 적용해 근거 적합성을 재판단하고, 부족 시 재검색 또는 Abstain으로 Hallucination 완화 구조를 적용했습니다.

03 기술명: Optimization / Dispatching + DES

선택 이유:

단순 조회를 넘어 작업 우선순위, 피킹·적치 순서, 자원 배정, 디스패칭까지 의사결정이 필요해 최적화/시뮬레이션 엔진을 반영했습니다.

구현 포인트:

현재는 Rule·Heuristic 기반 Dispatching을 적용했고, DES What-if 시뮬레이션으로 처리량·자연·자원 활용도를 검증합니다. MILP는 향후 확장 방향입니다.

핵심 기술 난제 (Hurdle)

자연어 질문을 단순 답변으로 끝내지 않고, 올바른 업무 Tool과 정책 근거를 선택하면서도 근거 부족 시 잘못된 답변이나 위험한 상태 변경을 방지해야 했습니다.
단순 ReAct 흐름만으로는 잘못된 Tool 호출, 근거 부족 답변, 승인 없는 상태 변경 위험을 동시에 제어하기 어려웠습니다.

엔지니어링 접근 방법



1 Adaptive Agent Orchestration

- 26개 Intent로 질문 분류
- Parameter 누락 시 재질문
- Tool 실행 후 Verifier로 결과 검증
- 상태 변경은 Draft와 Approval Gate로 분리



2 근거 충분성 기반 RAG

- FAISS로 후보 문서 검색
- PRISM 방식으로 관련성-근거 구간 재정렬
- Sufficiency Judge로 답변 가능 여부 판정
- 부족 시 최대 2회 재검색 후 Abstain



3 안전한 자동운영 (Blackboard Control Loop)

- Event 기반 Agent 제안
- Simulation Gate와 Policy Check
- Idempotency Key와 Lock 적용
- Pre-check → Transaction → Post-check → Audit



핵심 기술 스택

- LangGraph / FastAPI / Python 3.11 / SQLite
- FAISS / PRISM / Sufficiency Judge / Azure OpenAI 호출 Gateway
- SimPy DES / Forecast / Rule-Heuristic Dispatching / SSE / Vanilla JS SPA

결과 및 성과 (Key Results)

최대 2회
재검색

근거 부족 대응

- 근거가 부족한 경우 최대 2회까지 재검색을 수행해 답변 신뢰성 향상
- 부족 시 Abstain하여 잘못된 답변 방지

4단계
안전 실행

Pre-check → Transaction → Post-check → Audit

- 실행 전 검증부터 실행, 사후 검증, 감사 기록까지 단계적으로 보장
- 모든 변경은 투명하게 추적

3중
실행 통제

Approval Gate + Policy Check + Lock

- Approval Gate로 승인 여부 결정
- Policy Check로 정책 위반 여부 검증
- Lock으로 중복 실행과 충돌 방지